

PS> VSCode Everywhere

Set Up Once Use Anywhere



PS> Hi, I'm Constantin



Microsoft
Most Valuable
Professional

10 years of
PowerShell
experience



Speaker at PSConf EU 2026



Organizer, PSUG Inn-Salzach



Blog: the-itguy.de



GitHub: github.com/constantinhager



Hobbies: IT and singing

Agenda

01 The Real-World Problem

02 Dev Container in VS Code

03 GitHub Codespaces

04 Better Together

05 Live Demos

The Real-World Problem



"Works on My Machine"

Bugs that only reproduce on one dev's setup — impossible to share or fix



Slow Onboarding

New team members spend 1–3 days on setup before writing a single line of code



Tool Version Conflicts

PowerShell 5.1 vs 7.x — switching projects silently breaks scripts and modules



Missing Extensions

Everyone has different VS Code extensions — linting, formatting, IntelliSense all behave differently



Module Hell

Required modules missing, wrong version, or installed globally — breaking other scripts



CI/CD Drift

Pipeline uses different tool versions than local dev — passing locally, failing in CI

The "Anywhere" Challenge



Any Device

Laptop, desktop, home PC — the environment must follow the dev, not the hardware



Any Location

Office, home, conference — full productivity from wherever you have a connection



Any OS

Windows or macOS host — the dev container abstracts away the underlying platform



Zero Local Setup

No SDKs, modules, or tools to install — open VS Code and you're ready to code



Instant Onboarding

New team member or borrowed laptop — productive in minutes, not days



VS Code is the Key

Already everywhere: Windows, macOS, browser — the missing piece is a consistent environment

Dev Containers: What & How



Defined in Code

devcontainer.json lives in your repo — the environment is version-controlled like any other code



Runs in Docker / Podman

VS Code connects via the Dev Containers extension — your code runs inside an isolated container



Full VS Code Experience

Extensions, settings and themes defined in the container — consistent for every team member



Any Base Image

Use [mcr.microsoft.com/devcontainers/dotnet images](https://mcr.microsoft.com/devcontainers/dotnet-images) — PowerShell, .NET and more, out of the box.



Post-Create Commands

Run Install-Module, dotnet restore automatically when the container starts



Port Forwarding

Expose local ports from inside the container transparently — test web apps and APIs as normal

Dev Containers: Why It Works



Identical Environments

Windows, macOS, or Linux host — every dev gets the exact same container, every time



Team Onboarding in Minutes

Clone, open in VS Code, reopen in container — fully productive with no setup guide needed



Extensions as Code

Required VS Code extensions are listed in devcontainer.json — installed automatically for everyone



Right PowerShell Version

Pin PS 7.4 or 5.1 per project — switching repos never breaks your global PowerShell again



Modules Pre-Installed

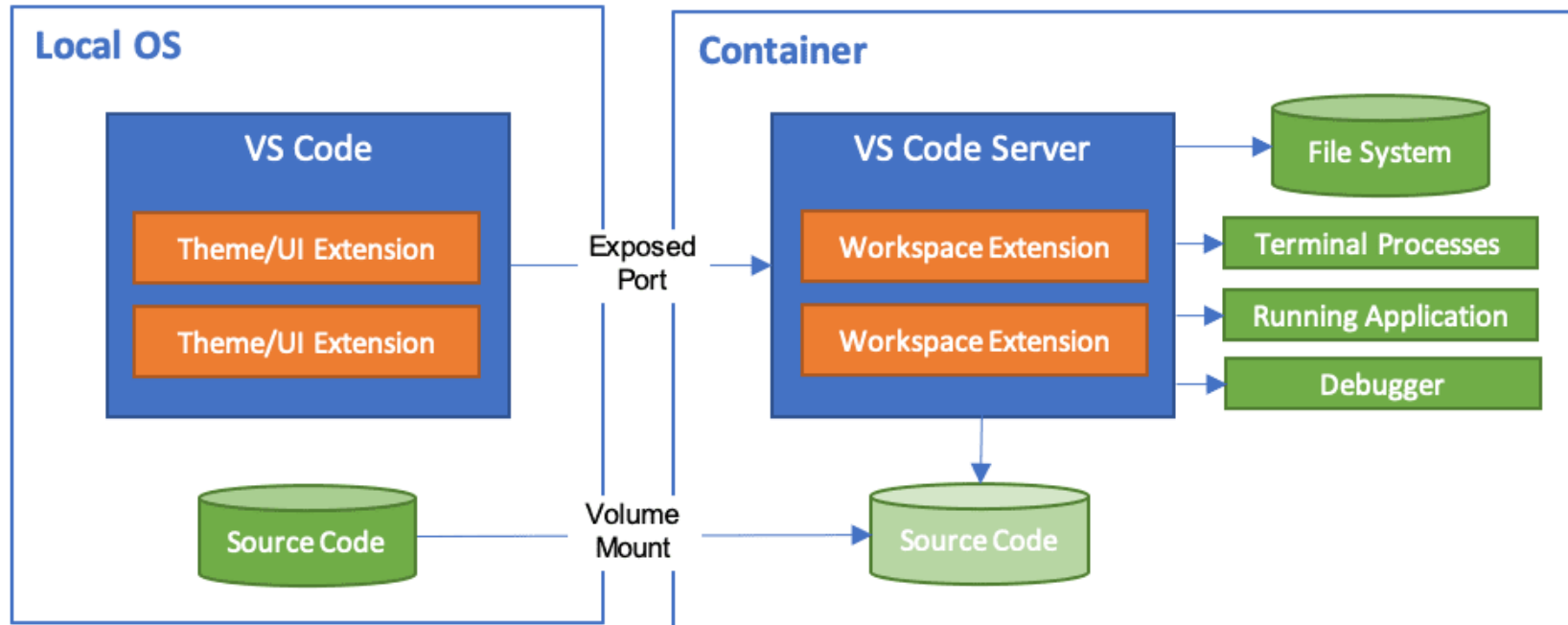
Run Install-Module in postCreateCommand — every dev starts with the exact right modules ready



Works Everywhere

Same container config powers local VS Code, GitHub Codespaces, and CI/CD pipelines

Architecture Dev Containers



GitHub Codespaces: What & How



Cloud-Hosted VS Code

A full VS Code environment in the cloud — same editor, same shortcuts, same feel as local



Powered by devcontainer.json

devcontainer.json is the single source of truth — Codespaces reads it automatically on start



Browser or Desktop Client

Use VS Code in any browser or connect your local VS Code client — no install required either way



Scalable Compute

Choose 2-core to 32-core VMs — scale up for builds, scale down to save cost



Persistent Workspaces

Your files, terminal history, and running processes persist between sessions — just stop and resume



GitHub-Native

Launches directly from any GitHub repo — one click from the repo page, no local clone needed

GitHub Codespaces: Why It Works



True "Use Anywhere"

Same VS Code, same config, same terminal — running in the cloud from any device or OS



No Local Prerequisites

No Docker, no SDKs, no PowerShell install needed — just a browser or VS Code client



Instant Environment

From zero to a fully configured PowerShell dev environment in under 2 minutes



Consistent with Local Dev

The same devcontainer.json powers local Dev Containers and Codespaces — no config drift



Team Standardization

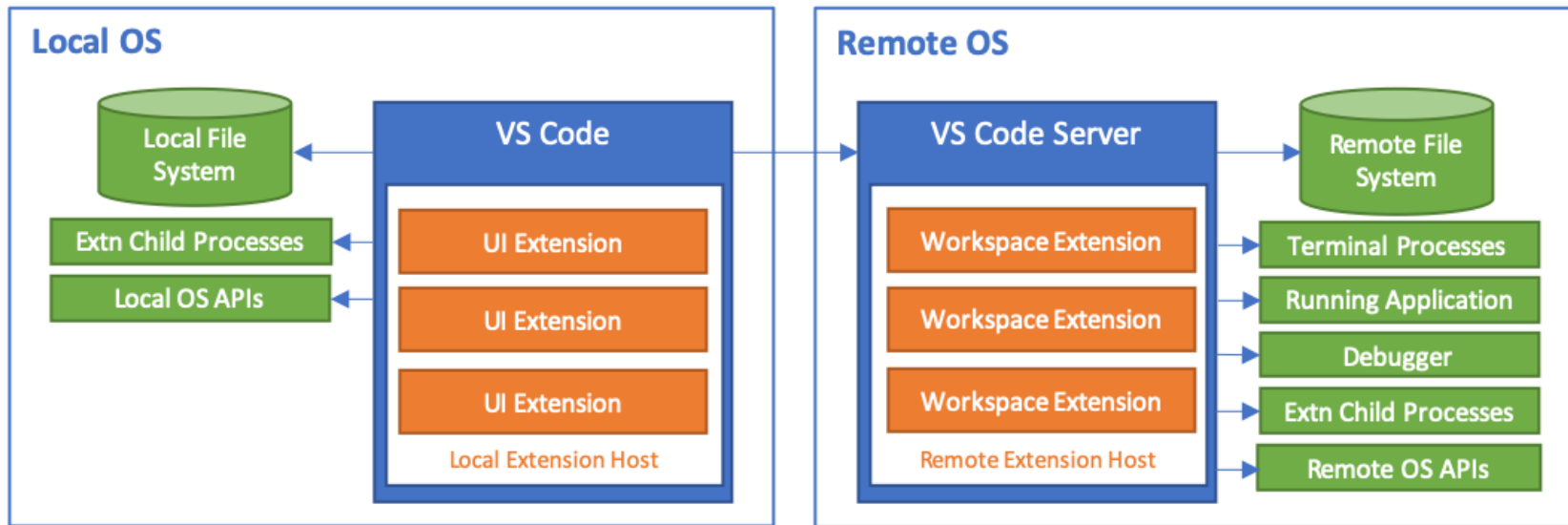
Every contributor gets identical tools, modules, and extensions — no more environment surprises



Cost Controlled

Pay only for active compute time — stop the Codespace when done, resume exactly where you left off

Local VSCode with GH Codespaces



Better Together: One Config



Local Dev Containers

VS Code + Docker on your own machine — full offline capability, full control, zero cloud cost



GitHub Codespaces

Same config, cloud-hosted — open from any browser or VS Code client with no local install



CI/CD Pipelines

GitHub Actions runs inside the same devcontainer — build and test in the exact dev environment



One devcontainer.json

One file drives all modes — update it once and every environment updates automatically



No Mode Switching Cost

Move from local to cloud to CI seamlessly — your terminal history, extensions, and tools follow you

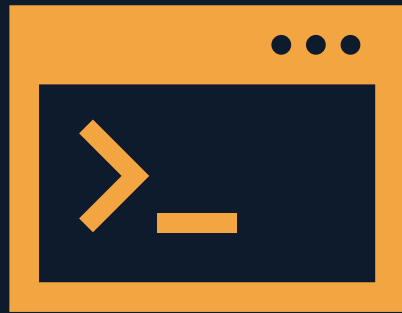


Environment as Code

Reviewed in PRs, versioned in Git, rolled back if broken — the environment is a first-class citizen

PS> Demos

We love Demos!



The Promise — Delivered



Lost Your Laptop?

Open a Codespace from any browser



Broke Your OS?

Your dev environment lives in the container, not on your host machine



New Team Member Today?

Fork, clone, open in container



Conference on a Thin Client?

Any machine with a browser runs your full PowerShell environment via Codespaces — no install



Build Fails Only in CI?

CI runs in the same devcontainer as local. Same tools, same versions, always



The Single Source of Truth

devcontainer.json in the repo — one file governs every developer, every machine, every pipeline

PS> Q&A



PS> Thank you for having me

Let's talk PowerShell, Azure and community.

the-itguy.de · github.com/constantinhager

Constantin Hager · Microsoft MVP